

Enhancing Story Point Estimation in Software Projects Using Transformer-Based Embeddings and Ensemble Regression Models

1st Darren Cornelius Citra Wijaya
Computer Science Department,
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
darren.wijaya001@binus.ac.id

2nd Auryn Larissa Sabrina
Computer Science Department,
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
auryn.sabrina@binus.ac.id

3rd Meiliana
Computer Science Department,
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
meiliana@binus.edu

Abstract—Story point plays a vital role in agile software development by helping teams assess the required effort to complete user stories. Accurate estimation can improve planning and resource allocation, yet there are a lot of challenges that persist due to the subjective and complex nature of software tasks. Traditional machine learning models have also tried to address this, yet they're limited in understanding the semantic relationship between natural language within user stories. To address these gaps, we propose combined methods through transformer-based embeddings like BERT and RoBERTa, and these embeddings have proven significant capabilities in capturing the semantic relationship between words. Our study compares multiple text embedding techniques like TF-IDF, Word2Vec, BERT, and RoBERTa with ensemble regression models, including Random Forest, XGBoost, and LightGBM, to identify effective approaches for automating story point estimation. The result suggests that transformer-based embeddings combined with XGBoost and LightGBM achieve the lowest Mean Absolute Error of 1.18 and 1.17 respectively. These findings highlight the effectiveness of improving estimation accuracy within agile methodologies.

Keywords—Story Point Estimation, Machine Learning, Ensemble Regression Models, Transformer Embeddings, Mean Absolute Error, Random Forest, XGBoost, LightGBM.

I. INTRODUCTION

Story points are widely used to estimate the effort involved in a user story [1]. Story points are important in any software or agile development project [2], as they help teams assess the amount of effort needed to complete tasks and manage resources effectively. Finding tools to automate story point estimation has become increasingly suitable, offering substantial potential to improve the estimation accuracy [3]. However, the software development industry still faces challenges with estimating development effort accurately, leading to potential delays and inefficiencies in project execution. To address these challenges, some research has been looking for automation technologies for user story point or effort estimation [4]. Accurate user story point or effort estimation can enhance the development planning by allowing for the optimal assignments of user stories and developers. Automatic story point estimation is needed when estimating issues becomes complex and developers may produce inaccurate results, especially when they do not have enough experience, making automatic approaches valuable to support and improve the estimation process [5]. Various techniques and algorithms have been proposed to automate

story point estimation. The ensemble-based approach has shown great promise in enhancing the accuracy of effort estimation [6].

Recent research demonstrated that the use of ensemble learning techniques like Gradient Boosting Tree (GBT) models has shown promising results in improving story point estimation [3]. This shows the strong performance in handling structured data, making them suitable candidates for effort prediction tasks in software development. TF-IDF is commonly used because of its clarity and explainability [7]. Transformer embedding like BERT is conceptually simple and highly effective, achieving advanced performance over some NLP tasks [8]. More research and innovation are needed to explore ensemble learning and transfer learning techniques to enhance the user story point or effort estimation models.

To address these gaps, this study proposes a comparison of various text embedding techniques, such as TF-IDF, Word2Vec, BERT, and RoBERTa, combined with ensemble regression models. By evaluating the combinations of these two components, our goal is to identify the most effective and accurate approach for estimating the story point with MAE. Through this evaluation, we aim to provide insights into the practical effectiveness of modern NLP tools in enhancing software effort estimation automation.

II. RELATED WORKS

Effort estimation is known for its uncertainty but crucial for project planning [9]. Approaches these days rely on applying pre-trained embedding models and deep learning for text regression [2]. A combination of machine learning models (ensemble) has improved the accuracy compared to individual models [10]. Ensemble models show a better result compared to individual models [11] because they provide more accurate estimations [12].

Another study [3] utilizes SBERT for feature extraction combined with GBT, which is one of the ensembles learning techniques and achieved 2.15 for MAE. Similarly, [6] developed an ensemble-based method comprising SVM, ANN, KNN, DT, and RR, after finding that none of the seven individual predictive algorithms consistently outperforms. Another research [1] proposed a deep learning-based

prediction model combining LSTM and RHN, the combined model demonstrates better performance compared to using LSTM or RHN individually.

Moreover, another research [13] used TF-IDF to extract semantic features from issue reports for estimating story points. Similarly, [14] used TF-IDF to convert user stories into numerical feature vectors, enabling the K-Means algorithm to perform effective clustering in Agile requirement engineering. Furthermore, other research [15] also used TF-IDF for feature extraction alongside BERT for effort estimation, and the results suggest that there is no significant difference between the two approaches.

There's also research conducted [16] that shows traditional machine learning models like Random Forest, SVM, and Linear SVM are more efficient and accurate in structured datasets. Large Language Models (LLMs) like BERT and GPT-2 may offer advantages in scenarios requiring deep language understanding for agile automating story point estimation. Random forest is a good technique for software development effort estimation [17]. Random Forest and linear regression demonstrated strong performance in software effort estimation, achieving the best R^2 scores and low RMSE, especially with correlation-based feature selection [18].

Further research [19] proposed GPT2SP, a Transformer-based approach for Agile software effort or story point estimations, the result shows the median of MAE is 1.16. Another research [20] proposes an Artificial Neural Network (ANN) to estimate project effort and in [11], ANN was used as part of a heterogeneous ensemble model with Use Case Point (UCP) and Expert Judgment (EJ), which outperforms individual models but is complex due to the integration of diverse parameters and architectures. Other approaches, such as fuzzy logic models, have also been applied in effort estimation, but are very sensitive to outliers in predicting software estimation effort [21].

Based on reviewed studies, story point estimation focuses on pre-trained text techniques like BERT and SBERT, with modern regression models like GBT and Artificial Neural Network. Ensemble approaches consistently outperformed individual models due to their techniques in combining multiple algorithms. Additionally, transformer-based models like GPT-2 have shown exceptional performance, surpassing several traditional methods in text regression. The most promising direction for accurate and automated user story point estimation relies on combining a pre-trained model with the predictive power of ensemble machine learning models.

III. METHODOLOGY

The methodology of this research was to address the limitations identified in prior studies on effort or story point estimation in agile projects [22]. Conventional methods often struggle to capture sentence relation or meaning in user story descriptions, while ensemble-based regressors have proved effective in story point estimation tasks [23].

Recent advances in NLP, particularly transformer-based embeddings (e.g., BERT, GPT) have demonstrated strong performance in providing contextual information from sentences. Our study integrates text embedding techniques with ensemble regression models to improve the accuracy of story point prediction.

Specifically, four embedding strategies were selected to represent both conventional and transformer-based approaches. These strategies are then paired with ensemble regressors, which have been previously shown to perform well in software effort estimation. Mean Absolute Error is used as the primary evaluation metric, in line with previous research where MAE is widely used to measure prediction accuracy in effort estimation.

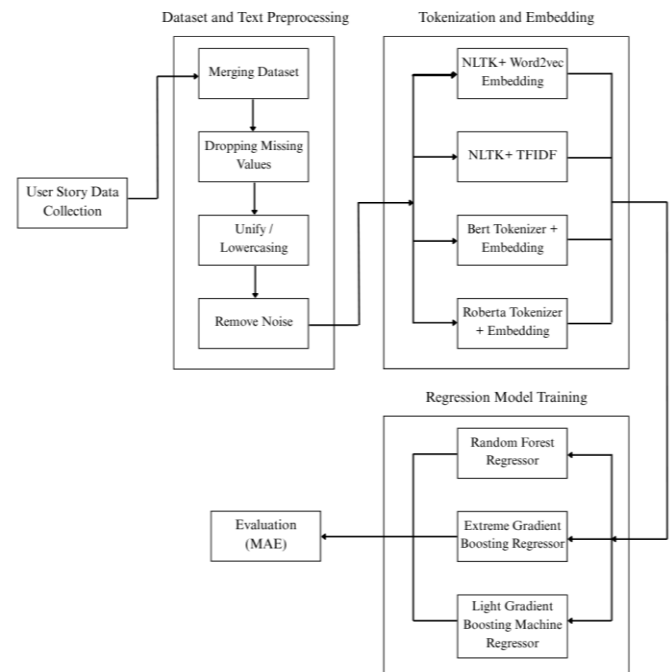


Fig 1. Methodology Workflow

A. Dataset

The dataset that is used in this research was sourced from open-source Agile projects [24] which consists of: Aurora, Spring XD, Usergrid, and Mesos. These datasets were collected in CSV and merged into a single dataset to ensure greater diversity and volume. After merging, the final dataset consists of 3544 rows. The dataset contains several missing values, which require preprocessing steps. Not all attributes in the dataset (listed in Table I) are going to be used for model training.

In this research, *story point* serves as target variables to be predicted. Each entry or user story in the dataset is labeled with a story point value that indicates the development effort estimated by the Agile team. For example, small values (e.g., 1, 2, or 5) often imply simple tasks with low complexity, while larger values (e.g., 10, 13, or higher) represent complex stories.

TABLE I. DATASET ATTRIBUTES

Attribute Name	Data Type
key	object
issueType	object
sprint	int64
status	object
summary	object
description	object
storyPoint	float64
priority	float64
watchcount	float64
fixVersions	float64
affectedVersions	float64
assignee	object
creator	object
reporter	object
commentCount	float64
votes	float64
issueLinks	float64
blockedBy	float64
blocks	float64
dependedOnBy	float64
dependedOn	float64
subtasks	float64

B. Preprocessing

The first step involves addressing missing values in the dataset, specifically on ‘storyPoint’ and ‘description’. The missing values on ‘storyPoint’ were imputed using the median of remaining available story points. Median was selected over mean because it’s less sensitive towards float numbers and better represents the central tendency of skewed distributions. Additionally, ‘description’ missing entries were filled with the placeholder string “No description”. This approach was chosen because we’re preserving the structural integrity of the dataset while ensuring that every record has a textual component for modeling.

We also dropped unnecessary columns and only selected a few columns. This approach is entirely to streamline the dataset and prevent unnecessary noise in the modelling process. After addressing missing data and dropping columns, the text is converted to lowercase to ensure case-insensitive processing. Following this, embedded URLs and hyperlinks were removed using regular expressions. This is important because URLs contain non-informative tokens that only increase string lengths without providing semantic value. Moreover, all non-alphanumeric characters were removed from the text, including punctuation, special

characters, and non-English characters. This step ensures that the tokenization step would yield meaningful semantics.

Tokenization is performed using the NLTK library. This function splits each text into individual word tokens for embedding techniques. While traditional tokenization works well for embedding approaches like TF-IDF and Word2Vec, advanced pre-trained models like BERT and RoBERTa rely on their own specialized tokenizers, which utilize sub word segmentation and byte-level encoding. Despite having differences, our main goal for all tokenization techniques remains the same, which is breaking down text into smaller units that can be embedded.

C. Tokenization and Embedding Techniques

This research experimented with four different tokenization and embedding techniques to assess their impact on regression performance. The first approach combined NLTK tokenization with Word2Vec [25]. These tokens were mapped to a vector representation using the genism Word2Vec model, like GloVe. To obtain a fixed-sized representation, vectors of all tokens were averaged, resulting in a single dense vector per sample.

Our second approach involves NLTK tokenization in combination with TF-IDF vectorization [26]. After tokenizing, TF-IDF (specifically from Scikit-learn) was applied to convert text into a sparse matrix. This technique assigns weight to each word based on its frequency over the corpus, emphasizing informative terms and reducing commonly occurring but less meaningful words.

The third method application involves the BERT tokenizer [27] and embedding. BERT tokenizer (from HuggingFace Transformers library) tokenizes input text into sub words that align with BERT vocabulary. These tokens then passed through a pre-trained BERT model to generate contextual embeddings. The final representation for each text is extracted either from the CLS token or by averaging the last hidden states of all tokens.

Similarly, the fourth approach involves RoBERTa tokenizer [27] and embedding. Like BERT, and RoBERTa is also a transformer-based language model but employs a dynamic masking strategy. Texts are tokenized with RoBERTa’s tokenizer and passed through to its model to obtain embeddings. These embeddings are then extracted using CLS token or mean pooling of token embeddings, capturing semantic information more effectively.

D. Model Training and Accuracy Report

After text conversion into numerical form using the techniques mentioned, the resulting vectors were used to train regression models to predict numerical values from story points. We chose three ensemble-based models, which are Random Forest Regressor, Extreme Gradient Boosting Regressor, and Light Gradient Boosting Machine Regressor. These models were selected because of their ability to capture non-linearity and handle high-dimensional features. The data was split into training and test sets with an 80:20 ratio. The features were then normalized using Robust Scaler. This

scale is robust towards outliers, hence the name, and ensures that extreme values don't affect the learning process.

Random Forest regressor is an ensemble model that combines multiple decision trees using random subsets of data and aggregates their outcomes to generate the final prediction, which helps improve accuracy and generalization. [28]. With 100 estimators and a maximum depth of ten, the model maintains a balance between complexity and overfitting. We also use two minimum split samples and one minimum leaf sample to allow the model to capture detailed patterns in the data.

Gradient boosting models like XGBoost and LightGBM are scalable and efficient ensemble methods that offer strong predictive accuracy with fast training performance. They build trees sequentially, where each new tree attempts to minimize the errors of the previous ones [29]. With similar configurations like Random Forest regressor and 0.1 learning rate, both models are designed to optimize the performance while maintaining training efficiency.

Overall, these ensemble methods complement each other. Random Forest regressor offers strong baseline performance, while XGBoost provides optimization and robustness, and LightGBM excels in speed and scalability, making them well-suited for regression tasks like user story point estimation or software effort estimation.

To evaluate prediction accuracy, we use Mean Absolute Error as an error metric. MAE is calculated by taking the absolute difference between the predicted and actual story points, then averaging across all samples. Lower MAE values imply higher accuracy, meaning the model's predictions are closer to the estimates originally assigned by the Agile team. MAE was selected because it is intuitive, easy to understand, and less sensitive to extreme outliers compared to RMSE, and is widely used in software effort estimation research.

IV. RESULT AND DISCUSSION

Our primary evaluation metric to assess the model, as mentioned before, was MAE, which is calculated by the average absolute difference between predicted and actual story point values. Lower Mean Absolute Error values imply better predictive accuracy. Table II summarizes the results of the best MAE Performance.

From all the embedding and tokenization methods, Extreme Gradient Boosting Regressor consistently gives the lowest Mean Absolute Error values, closely followed by Light Gradient Boosting Machine Regressor. This also indicates that XGBoost and LightGBM are both gradient boosting frameworks that capture complex patterns in the data. This also suggests that boosting techniques can capture textual features, particularly in capturing deep semantic or contextual relationships among the features.

When compared with traditional embedding approaches, Word2Vec outperforms TF-IDF with an MAE of 1.24 using LightGBM. Poor performance by TF-IDF is suggested by its fundamental limitation, it's a bag-of-words representation

that captures only TF-IDF, without any knowledge of the context, order, or semantic similarities. As a result, important words in the user stories requirements were ignored.

TABLE 2. MEAN ABSOLUTE ERROR RESULTS. BEST MAE PERFORMANCE IS SHOWN IN BOLD AND THE SECOND-BEST IS SHOWN IN UNDERLINE

Tokenization + Embedding Technique	Model	Mean Absolute Error (MAE)
NLTK + TF-IDF	Random Forest	1.29
	XGBoost	1.21
	LightGBM	1.30
NLTK + Word2Vec	Random Forest	1.27
	XGBoost	1.26
	LightGBM	1.24
BERT	Random Forest	1.31
	XGBoost	<u>1.18</u>
	LightGBM	<u>1.18</u>
RoBERTa	Random Forest	1.30
	XGBoost	1.17
	LightGBM	1.22

In contrast, Word2Vec learns word embeddings that encode semantic similarity, which alone allows the model to understand word relationships. These embeddings provide general semantic knowledge that improves the quality of feature representation used in the regression task. Despite being lightweight in terms of computational resources compared to transformer-based models, Word2Vec provides a strong balance between performance and computational efficiency.

BERT and RoBERTa embeddings give the strongest overall performance, particularly when used with a gradient boosting regressor. Lowest MAE was achieved by RoBERTa with a score of 1.17 when using XGBoost, followed closely by BERT paired with both gradient boosting regressors at 1.18. These results indicate the effectiveness of transformer-based models in capturing contextual information from story descriptions.

These findings imply the development of machine learning tools in Agile projects. While prior studies with SBERT with Gradient Boosting Trees reported higher error (MAE of 2.15) and traditional approaches based on TF-IDF often struggled to capture semantic meanings.

Another study highlighted the effectiveness of traditional models like SVM and Linear SVM in structured datasets, reporting strong efficiency in effort estimation tasks and achieving high R^2 scores and low RMSE. However, while these models perform well in scenarios with structured numerical features, their capacity to understand contextual meaning from textual data is limited. Our findings extend this branch by showing when the data source is highly textual.

This comparison highlights that ensemble models such as XGBoost and LightGBM are effective when paired with transformer-based embeddings. While Random Forest has been shown to perform well in structured datasets, it's less

effective when dealing with heavy sentence data like Agile user stories.

Despite these promising results, the dataset used originates from a limited number of open-source Agile projects, which may not fully capture the variance and noise of industrial-scale datasets. Additionally, our experiments focused on regression ensemble models as deep neural regression architectures were not explored. Furthermore, story points themselves are subjective, which could introduce noise into training and evaluation.

V. CONCLUSION

Our study observed the impact of different text tokenization, embedding, and ensemble regression models on the accuracy of estimating story points in agile software development. With a comprehensive evaluation of traditional methods like TF-IDF along with BERT, our findings revealed that contextual embedding consistently outperformed traditional ones. Among all models, XGBoost demonstrated the best performance across all regressors, achieving low Mean Absolute Error when paired with RoBERTa. We conclude that the gradient boosting method is effective in capturing complexity and the semantic representation of user stories.

We also conclude that transformer-based embeddings combined with tree-based ensemble learners offer an efficient solution in automating story point estimation. This combination captures non-linear patterns within data, providing an accurate approach to story estimation. As agile teams rely on data-driven tools to support sprint planning, the integration of pre-trained models like BERT can enhance the reliability in estimating story points for new user stories. Future research can expand by incorporating additional features, experimenting with deep learning architectures, or applying the techniques used in real-world industries' datasets, and conducting a controlled computational-cost benchmark to provide quantitative guidance on resource trade-offs and alternatives that avoid GPU dependence.

DATA AVAILABILITY

This study utilizes an open-source dataset from Agile projects [24], which consists of: Aurora, Spring XD, Usergrid, and Mesos. These datasets were collected in CSV and merged into a single dataset to ensure greater diversity and volume. All data used in this study are openly accessible under their respective licenses.

AUTHORSHIP STATEMENT

Darren Cornelius Citra Wijaya and Auryn Larissa Sabrina: Data curation, developing a model, Result validation, and Paper Writing. Meiliana: Fundamental Ideation and Conceptualization, Methodology and Supervision.

REFERENCES

- [1] M. Choetkiertikul, H. K. Dam, T. Tran, T. Pham, A. Ghose, and T. Menzies, "A deep learning model for estimating story points," Sep. 2016, [Online]. Available: <http://arxiv.org/abs/1609.00489>
- [2] H. Phan and A. Jannesari, "Heterogeneous Graph Neural Networks for Software Effort Estimation," in International Symposium on Empirical Software Engineering and Measurement, IEEE Computer Society, Sep. 2022, pp. 103–113. doi: 10.1145/3544902.3546248.
- [3] B. Yalçın, K. Dinçer, A. G. Karaçor, and M. Ö. Efe, "Enhancing Agile Story Point Estimation: Integrating Deep Learning, Machine Learning, and Natural Language Processing with SBERT and Gradient Boosted Trees," Applied Sciences (Switzerland), vol. 14, no. 16, Aug. 2024, doi: 10.3390/app14167305.
- [4] A. Jadhav, S. K. Shandilya, I. Izonin, and M. Gregus, "Effective Software Effort Estimation Leveraging Machine Learning for Digital Transformation," IEEE Access, vol. 11, pp. 83523–83536, 2023, doi: 10.1109/ACCESS.2023.3293432.
- [5] E. Scott and D. Pfahl, "Using developers' features to estimate story points," in ACM International Conference Proceeding Series, Association for Computing Machinery, May 2018, pp. 106–110. doi: 10.1145/3202710.3203160.
- [6] O. Malgonde and K. Chari, "An ensemble-based model for predicting agile software development effort," Empir Softw Eng, vol. 24, no. 2, pp. 1017–1055, Apr. 2019, doi: 10.1007/s10664-018-9647-0.
- [7] I. Atoum and A. A. Ootom, "Enhancing Software Effort Estimation with Pre-Trained Word Embeddings: A Small-Dataset Solution for Accurate Story Point Prediction," Electronics (Switzerland), vol. 13, no. 23, Dec. 2024, doi: 10.3390/electronics13234843.
- [8] J. Devlin, M.-W. Chang, K. Lee, K. T. Google, and A. I. Language, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." [Online]. Available: <https://github.com/tensorflow/tensor2tensor>
- [9] H. Kassem, K. Mahar, and A. A. Saad, "Story Point Estimation Using Issue Reports With Deep Attention Neural Network," E-Informatica Software Engineering Journal, vol. 17, no. 1, 2023, doi: 10.37190/e-Inf230104.
- [10] J. T. H. Jose Thiago and A. L. I. Oliveira, "Ensemble Effort Estimation using dynamic selection," Journal of Systems and Software, vol. 175, May 2021, doi: 10.1016/j.jss.2021.110904.
- [11] S. S. Ali, J. Ren, K. Zhang, J. Wu, and C. Liu, "Heterogeneous Ensemble Model to Optimize Software Effort Estimation Accuracy," IEEE Access, vol. 11, pp. 27759–27792, 2023, doi: 10.1109/ACCESS.2023.3256533.
- [12] Y. Mahmood, N. Kama, A. Azmi, A. S. Khan, and M. Ali, "Software effort estimation accuracy prediction of machine learning techniques: A systematic performance evaluation," Softw Pract Exp, vol. 52, no. 1, 2022, doi: 10.1002/spe.3009.
- [13] S. Porru, A. Murgia, S. Demeyer, M. Marchesi, and R. Tonelli, "Estimating story points from issue reports," in ACM International Conference Proceeding Series,

- Association for Computing Machinery, Sep. 2016. doi: 10.1145/2972958.2972959.
- [14] B. Kumar, U. K. Tiwari, D. C. Dobhal, and H. S. Negi, "User Story Clustering using K-Means Algorithm in Agile Requirement Engineering," in *Proceedings of International Conference on Computational Intelligence and Sustainable Engineering Solution, CISES 2022*, Institute of Electrical and Electronics Engineers Inc., 2022, pp. 1–5. doi: 10.1109/CISES54857.2022.9844390.
- [15] M. Alhamed and T. Storer, "Evaluation of Context-Aware Language Models and Experts for Effort Estimation of Software Maintenance Issues," in *Proceedings - 2022 IEEE International Conference on Software Maintenance and Evolution, ICSME 2022*, Institute of Electrical and Electronics Engineers Inc., 2022, pp. 129–138. doi: 10.1109/ICSME55016.2022.00020.
- [16] L. Alexander and R. Jayadi, "MACHINE LEARNING FOR STORY POINT ESTIMATION: DO LARGE LANGUAGE MODELS OUTPERFORM TRADITIONAL METHODS?," *J Theor Appl Inf Technol*, vol. 31, no. 20, 2024, [Online]. Available: www.jatit.org
- [17] Z. Abdelali, H. Mustapha, and N. Abdelwahed, "Investigating the use of random forest in software effort estimation," in *Procedia Computer Science*, Elsevier B.V., 2019, pp. 343–352. doi: 10.1016/j.procs.2019.01.042.
- [18] Meharunnisa, M. Saqlain, M. Abid, M. Awais, and Ž. Stević, "Analysis of Software Effort Estimation by Machine Learning Techniques," *Ingenierie des Systemes d'Information*, vol. 28, no. 6, pp. 1445–1457, Dec. 2023, doi: 10.18280/isi.280602.
- [19] M. Fu and C. Tantithamthavorn, "GPT2SP: A Transformer-Based Agile Story Point Estimation Approach," *IEEE Transactions on Software Engineering*, vol. 49, no. 2, 2023, doi: 10.1109/TSE.2022.3158252.
- [20] B. Şengüneş and N. Öztürk, "An Artificial Neural Network Model for Project Effort Estimation," *Systems*, vol. 11, no. 2, Feb. 2023, doi: 10.3390/systems11020091.
- [21] A. B. Nassif, M. Azzeh, A. Idri, and A. Abran, "Software development effort estimation using regression fuzzy models," *Comput Intell Neurosci*, vol. 2019, 2019, doi: 10.1155/2019/8367214.
- [22] P. Pospieszny, B. Czarnacka-Chrobot, and A. Kobylinski, "An effective approach for software project effort and duration estimation with machine learning algorithms," *Journal of Systems and Software*, vol. 137, 2018, doi: 10.1016/j.jss.2017.11.066.
- [23] E. Rodríguez Sánchez, E. F. Vázquez Santacruz, and H. Cervantes Maceda, "Effort and Cost Estimation Using Decision Tree Techniques and Story Points in Agile Software Development," *Mathematics*, vol. 11, no. 6, 2023, doi: 10.3390/math11061477.
- [24] R. Koralage, "Agile Scrum Sprint Velocity Dataset," GitHub, 2022. [Online]. Available: <https://github.com/RandulaKoralage/AgileScrumSprintVelocityDataSet>
- [25] E. M. Dharma, F. Lumban Gaol, H. Leslie, H. S. Warnars, and B. Soewito, "THE ACCURACY COMPARISON AMONG WORD2VEC, GLOVE, AND FASTTEXT TOWARDS CONVOLUTION NEURAL NETWORK (CNN) TEXT CLASSIFICATION," *J Theor Appl Inf Technol*, vol. 31, no. 2, 2022, [Online]. Available: www.jatit.org
- [26] R. N. Rathi and A. Mustafi, "The importance of Term Weighting in semantic understanding of text: A review of techniques," *Multimed Tools Appl*, vol. 82, no. 7, pp. 9761–9783, Mar. 2023, doi: 10.1007/s11042-022-12538-3.
- [27] C. Özkurt, "Comparative Analysis of State-of-the-Art Q A Models: BERT, RoBERTa, DistilBERT, and ALBERT on SQuAD v2 Dataset," *Chaos and Fractals*, Jul. 2024, doi: 10.69882/adba.chf.2024073.
- [28] H. A. Salman, A. Kalakech, and A. Steiti, "Random Forest Algorithm Overview," *Babylonian Journal of Machine Learning*, vol. 2024, pp. 69–79, Jun. 2024, doi: 10.58496/bjml/2024/007.
- [29] C. Bentéjac, A. Csörgő, and G. Martínez-Muñoz, "A comparative analysis of gradient boosting algorithms," *ArtifIntell Rev*, vol. 54, no. 3, pp. 1937–1967, Mar. 2021, doi: 10.1007/s10462-020-09896-5.